

Relatório de Implantação e Testes

AWS RDS, Spring Boot e Validação de Banco de Dados

**JOSÉ ARTHUR CALIXTO DA ROCHA COSTA
JOSEPH ANTONY DOS SANTOS LEITE
LUAN DANIEL SANTANA COUTO**

[07 de Julho de 2026]

2. Objetivo

Este documento tem por objetivo formalizar e detalhar o processo de migração, configuração e validação do ambiente de banco de dados utilizando o serviço gerenciado **AWS RDS (Relational Database Service)**, integrado a uma aplicação construída sobre o ecossistema **Spring Boot**.

O foco central reside em garantir a consistência física e lógica dos dados após a importação de um snapshot/dump estrutural, o ajuste de integridade sequencial, bem como a validação ponta a ponta dos fluxos de criação, leitura, atualização e exclusão (CRUD) das entidades fundamentais do sistema.

3. Configuração da AWS RDS

3.1. Instância criada

Abaixo encontra-se a especificação e o registro visual da instância do banco de dados provisionada na infraestrutura da Amazon Web Services. A configuração assegura alta disponibilidade, segurança de rede via Security Groups restritivos e o dimensionamento correto de hardware para a carga de trabalho prevista.

database-1

[Modificar](#)[Ações ▼](#)

Resumo

Identificador de banco de dados database-1	Status ✔ Disponível	Função Instância	Mecanismo PostgreSQL
CPU 3.32%	Classe db.t4g.micro	Atividade atual 0.00 sessões	Região e AZ us-east-1a

Recomendações
[3 Informativa](#)

3.2. Conexão e importação do dump

Comando utilizado para importar o dump oficial:

```
psql -h <database> -U postgres -d engenharia_dados -f universidade-dump-engadados.sql
```

```

engenharia_dados=> \c engenharia_dados
psql (18.2, server 18.3)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
You are now connected to database "engenharia_dados" as user "postgres".
engenharia_dados=> \dt universidade.*
      List of tables
 Schema | Name          | Type  | Owner
-----+-----+-----+-----
 universidade | alocao         | table | postgres
 universidade | cursa          | table | postgres
 universidade | curso          | table | postgres
 universidade | departamento   | table | postgres
 universidade | disciplina      | table | postgres
 universidade | estudante      | table | postgres
 universidade | horario        | table | postgres
 universidade | leciona        | table | postgres
 universidade | plano          | table | postgres
 universidade | professor      | table | postgres
 universidade | projeto        | table | postgres
 universidade | sala           | table | postgres
 universidade | semestre       | table | postgres
 universidade | turma         | table | postgres
 universidade | usuario        | table | postgres
 universidade | vinculo        | table | postgres
(16 rows)

```

3.3. Correção das sequences

Após a importação, as sequences foram ajustadas para os próximos IDs: .

```

SELECT setval('universidade.curso_idcurso_seq', (SELECT MAX(idcurso) FROM universidade.curso));
SELECT setval('universidade.vinculo_idvinculo_seq', (SELECT MAX(idvinculo) FROM universidade.vinculo));

```

3.4. Verificação dos dados

Auditoria pós-migração realizada para conferência quantitativa do volume de registros migrados versus a origem.

Entidade / Tabela	Quantidade de Registros Esperados	Quantidade Identificada no RDS	Status da Validação
Usuário	28	28	✓ OK
Estudante	13	13	✓ OK
Curso	5	5	✓ OK
Vínculo	13	13	✓ OK

4. Configuração do Spring Boot

4.1. Variáveis de ambiente

As credenciais foram configuradas via variáveis de ambiente

```
backend > src > main > resources > application.properties
1  spring.application.name=engenharia-dados
2
3  spring.datasource.url=${SPRING_DATASOURCE_URL}
4  spring.datasource.username=${SPRING_DATASOURCE_USERNAME}
5  spring.datasource.password=${SPRING_DATASOURCE_PASSWORD}
6
7  spring.jpa.hibernate.ddl-auto=none
8  spring.jpa.show-sql=true
9  spring.jpa.properties.hibernate.format_sql=true
10 spring.jpa.open-in-view=false
11
12 spring.flyway.enabled=false
13
14 server.port=8080
```

```
$env:SPRING_DATASOURCE_URL="jdbc:postgresql://<database>"
$env:SPRING_DATASOURCE_USERNAME="postgres"
$env:SPRING_DATASOURCE_PASSWORD="<senha>"
```

4.2. Log de inicialização mostrando a conexão com o RDS

A aplicação inicia e conecta-se ao RDS com sucesso

```
PS C:\Users\joarr\OneDrive\Documents\GitHub\ProjetoFinal-EngenhariaDeDados\CRUD Relacional - Parte1\backend> mvn spring-boot:run
2026-07-05T22:23:01.045-03:00 INFO 37908 --- [engenharia-dados] [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.40]
2026-07-05T22:23:01.101-03:00 INFO 37908 --- [engenharia-dados] [ restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-07-05T22:23:01.102-03:00 INFO 37908 --- [engenharia-dados] [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 996 ms
2026-07-05T22:23:01.222-03:00 INFO 37908 --- [engenharia-dados] [ restartedMain] o.hibernate.jpa.internal.util.LogHelper : HHH000204: Processing PersistenceUnitInfo [name: default]
2026-07-05T22:23:01.262-03:00 INFO 37908 --- [engenharia-dados] [ restartedMain] org.hibernate.Version : HHH000412: Hibernate ORM core version 6.6.13.Final
2026-07-05T22:23:01.291-03:00 INFO 37908 --- [engenharia-dados] [ restartedMain] o.h.c.internal.RegionFactoryInitiator : HHH000026: Second-level cache disabled
2026-07-05T22:23:01.490-03:00 INFO 37908 --- [engenharia-dados] [ restartedMain] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-07-05T22:23:01.514-03:00 INFO 37908 --- [engenharia-dados] [ restartedMain] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-07-05T22:23:03.611-03:00 INFO 37908 --- [engenharia-dados] [ restartedMain] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@50bbdeb
2026-07-05T22:23:03.614-03:00 INFO 37908 --- [engenharia-dados] [ restartedMain] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-07-05T22:23:04.231-03:00 INFO 37908 --- [engenharia-dados] [ restartedMain] org.hibernate.orm.connections.pooling : HHH10001005: Database info:
```

5. Testes CRUD

5.1. Scripts utilizados

Foram criados scripts PowerShell na pasta `scripts/`

- `test-usuario.ps1`
- `test-estudante.ps1`
- `test-curso.ps1`
- `test-vinculo.ps1`
- `test-all.ps1` (executa todos)

A função deles é de validar o CRUD em nossas tabelas.

5.2. Execução de todos os testes

O comando `.\test-all.ps1` foi executado e todos os testes passaram sem erros:

5.3. Resultados para cada entidade

Detalhamento do comportamento operacional por domínio do sistema:

Usuário

- Criado com CPF 3333333301
- Atualizado nome para "Usuario Teste Java Atualizado"
- Deletado com sucesso

Estudante

- Criado com CPF 3333333301
- Atualizado MC e ano de ingresso
- Deletado com sucesso

Curso

- Criado com ID 14
- Atualizado nome e turno
- Deletado com sucesso

Vínculo

- Criado com ID 14
- Atualizado status para "Cancelada"
- Deletado com sucesso

6. Validação no banco (consulta direta no RDS)

Para comprovar a persistência, foi realizada uma consulta direta no RDS após a criação de um estudante:

```

iaDeDados\CRUD Relacional - Parte1> $body = @{
>>     cpf = 3333333301
>>     nome = "Usuario Teste Java"
>>     dataNascimento = "2000-01-15"
>>     email = @"usuario.teste@email.com"
>>     telefone = @"79999999999"
>>     login = "usuario.teste.java"
>>     senha = "123456"
>> } | ConvertTo-Json

```

```

iaDeDados\CRUD Relacional - Parte1> Invoke-RestMethod -Uri "http://localhost:8080/api/usuarios" -Method Post -ContentType "application/json; charset=utf-8" -Body $body

```

```

cpf          : 3333333301
nome         : Usuario Teste Java
dataNascimento : 2000-01-15
email        : {usuario.teste@email.com}
telefone     : {79999999999}
login        : usuario.teste.java

```

Query

Query History

1

SELECT * FROM universidade.usuario WHERE cpf = '3333333301';

Data Output

Messages

Notifications

Showing rows: 1 to 1

Page No: 1 of 1

	cpf [PK] numeric (13)	nome character varying (100)	data_nascimento date	email character varying[]	telefone character varying[]	login character varying (45)	senha character varying (32)
1	3333333301	Usuario Teste Java	2000-01-15	{usuario.teste@email.co...	{79999999999}	usuario.teste.java	123456

07/07/2026 16:39:33
1
757 msec

Date
Rows affected
Duration

Copy
Copy to Query Editor

SELECT * FROM universidade.usuario WHERE cpf = '3333333301';

Messages

Successfully run. Total query runtime: 757 msec. 1 rows affected.

Remoção do Usuário:

```

iaDeDados\CRUD Relacional - Parte1\backend\scripts> Invoke-RestMethod -Uri "http://localhost:8080/api/usuarios/3333333301" -Method Delete

```

Query

Query History

1

SELECT * FROM universidade.usuario WHERE cpf = '3333333301';

Data Output

Messages

Notifications

07/07/2026 16:47:54

798 msec

Date

Rows affected

Duration

Copy

Copy to Query Editor

```
SELECT * FROM universidade.usuario WHERE cpf = '33333333301';
```

Messages

Successfully run. Total query runtime: 798 msec. 0 rows affected.

7. Conclusão

Todos os requisitos da Parte 1 foram atendidos:

- Banco PostgreSQL hospedado na AWS RDS.
- Importação do dump e correção das sequences.
- Aplicação Spring Boot conectada ao RDS.
- Testes CRUD validados para todas as entidades.
- Evidências documentadas